

# 18 · Guía para un nuevo desarrollador

Documentación de sistema

Versión analizada: **0.1.1+2**

Commit: **a7ebf2c**

Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build\_system\_documentation\_pdf.py**. El Markdown es la fuente: no editar este PDF.

## Contenido

---

1. Antes de nada: qué NO es este proyecto
2. Qué leer primero
3. Preparar el entorno
4. Ejecutar
5. Cómo está organizado el repositorio
6. Seguir un flujo completo
7. Dónde añadir cada cosa
8. Crear pruebas
9. Qué requiere especial cuidado
10. Convenciones del código
11. Convenciones de Git
12. Antes de abrir un pull request
13. Itinerario sugerido
14. Tareas apropiadas para empezar
15. A quién preguntar
16. Errores que ya cometió alguien

Escrita para alguien que llega hoy al proyecto y quiere ser útil esta semana.

## Antes de nada: qué NO es este proyecto

Aclararlo primero ahorra malentendidos:

- **no es un lector de QR con una advertencia añadida.** Es un sensor que explica antes de permitir actuar. Si una propuesta acelera la lectura a costa de la explicación, va contra el producto;
- **no tiene servidor.** No hay API que consumir ni backend que desplegar;
- **no puede decir «seguro».** Hay una prueba y un verificador que lo impiden;
- **no envía nada.** Cualquier llamada de red sería un cambio de contrato, no una funcionalidad.

## Qué leer primero

En este orden. Unas tres horas en total.

| Orden | Documento                                                | Tiempo | Para qué                           |
|-------|----------------------------------------------------------|--------|------------------------------------|
| 1     | <a href="#">../../README.md</a>                          | 15 min | Qué promete el producto            |
| 2     | <a href="#">01-system-overview.md</a>                    | 20 min | Qué hace y qué no                  |
| 3     | <a href="#">03-architecture.md</a>                       | 30 min | Cómo está organizado               |
| 4     | <a href="#">../rootcause/HEURISTICS.md</a>               | 30 min | Las 26 reglas, que son el producto |
| 5     | <a href="#">06-deep-code-explanation.md</a>              | 45 min | Por qué el código es así           |
| 6     | <a href="#">../quality/COMPATIBILITY-150-CONTRACT.md</a> | 15 min | Qué <b>no</b> se puede romper      |
| 7     | <a href="#">15-risks-and-technical-debt.md</a>           | 20 min | Qué ya se sabe que está flojo      |

## Preparar el entorno

```
git clone https://github.com/vladimiracunadev-create/rootcause-qr-inspector.git
cd rootcause-qr-inspector
flutter --version      # debe ser 3.44.7
python --version       # 3.12
flutter pub get
python tool/bootstrap.py --platforms android,web
python tool/validate_structure.py --require-lock
python tool/verify_rootcause_contract.py
flutter analyze --fatal-infos
flutter test
```

Si los siete comandos pasan, el entorno está listo.

**La sorpresa habitual:** android/, ios/ y web/ **no están en el repositorio.** Las genera el bootstrap. No las edites a mano: cualquier cambio ahí se pierde en la siguiente generación. Los ajustes de plataforma viven en tool/bootstrap.py.

## Ejecutar

```
flutter run          # dispositivo o emulador
flutter run -d chrome # demo web: sin PDF ni almacén seguro
```

## Cómo está organizado el repositorio

```
lib/
  main.dart          punto de entrada
  bootstrap*.dart    composición y arranque seguro
  app.dart           tema, bloqueo y navegación
  core/             infraestructura transversal
    investigation/  ← EL PRODUCTO ESTÁ AQUÍ
    security/       cifrado, rotación, adaptador de riesgo
    database/       Sembast y migraciones
    recovery/       aislamiento de registros
    ...            rendimiento, tema, idioma, diagnóstico
  models/           entidades inmutables
  services/         intérprete, repositorios, import/export, plataforma
  state/            tres stores ChangeNotifier
  features/         una carpeta por caso de uso

test/               88 casos
tool/               13 herramientas Python y shell
docs/               36 documentos
fixtures/ test_assets/ datos de regresión
```

Si solo puedes mirar una carpeta, que sea `lib/core/investigation/`.

## Seguir un flujo completo

Recorre esta cadena con el editor abierto; es la columna vertebral del sistema:

1. `lib/features/scanner/scanner_screen.dart` → `_handleCapture`
2. `lib/models/scan_record.dart` → `ScanRecord.fromBarcode`
3. `lib/services/content_interpreter.dart` → `ContentInterpreter.parse`
4. `lib/core/investigation/qr_investigation_engine.dart` → `analyze`
5. `lib/features/result/scan_result_sheet.dart` → `ScanRecordCard`
6. `lib/state/scan_store.dart` → `addAll`
7. `lib/services/history_repository.dart` → `upsertAll`
8. `lib/core/security/payload_cipher.dart` → `encryptToJson`

Ocho archivos. Después de recorrerlos entiendes el 80 % del sistema.

Para verlo en marcha sin cámara:

```
flutter test test/core/qr_investigation_engine_test.dart
```

## Dónde añadir cada cosa

| Quiero...                    | Va en...                                  | Y además debo...                             |
|------------------------------|-------------------------------------------|----------------------------------------------|
| Una regla nueva              | <code>qr_investigation_engine.dart</code> | Ver la lista de siete pasos más abajo        |
| Interpretar un formato nuevo | <code>ContentParser</code> + registro     | Decidir su <code>sensitive</code> y probarlo |
| Una pantalla nueva           | <code>lib/features/&lt;caso&gt;/</code>   | Pasar las dependencias por constructor       |

|                          |                                                                       |                                               |
|--------------------------|-----------------------------------------------------------------------|-----------------------------------------------|
| Un campo persistente     | El modelo + <code>toJson/fromJson</code>                              | Comprobar que un JSON antiguo sigue leyéndose |
| Un ajuste nuevo          | <code>AppSettings</code> + <code>SettingsRepository</code> + pantalla | Elegir un valor por defecto conservador       |
| Un límite de rendimiento | Donde se usa, como constante nombrada                                 | Documentarlo en <code>PERFORMANCE.md</code>   |
| Un texto visible         | Por ahora, literal en la pantalla                                     | Idealmente, <code>AppLocalizations</code>     |

## Añadir una regla: los siete pasos

Una regla no está terminada hasta que **los siete** están en el mismo cambio:

1. `qr_investigation_engine.dart`: `evaluate('mi-regla')` y su `add(...)`;
2. `qr_finding_text.dart`: título, explicación y recomendación;
3. `schemas/rootcause-qr-evidence.schema.json`: el id en `$defs.findingId.enum`;
4. `docs/rootcause/HEURISTICS.md`: fila en la tabla, con el id entre acentos graves;
5. `fixtures/qr/manifest.json`: un caso sintético con dominio reservado;
6. `test/core/qr_investigation_engine_test.dart`: prueba positiva y, si hay excepción legítima, negativa;
7. `QrInvestigationEngine.engineVersion`: subirla.

`python tool/verify_rootcause_contract.py` falla si olvidas cualquiera de los cuatro primeros. Y recuerda actualizar el número «26» donde aparezca.

## Crear pruebas

Convenciones observadas en el repositorio:

```
test('descripción en la lengua del archivo, en presente', () {
  // Arrange: el mínimo necesario
  // Act: una sola acción
  // Assert: una propiedad, no muchas
});
```

- los nombres describen **la propiedad**, no la función: «un respaldo no puede inyectar un veredicto», no «prueba de fromJson»;
- los comentarios explican **por qué** existe el caso;
- para la base de datos, `AppDatabase.openTemporary()` con `MemoryEncryptionKeyProvider`, y `addTearDown(database.close)`;
- en widgets con temporizadores reales, `tester.runAsync` y nunca `pumpAndSettle` sobre algo que anima siempre;
- fechas fijas (`DateTime.utc(2026, 8, 20)`) para que el resultado sea determinista.

## Qué requiere especial cuidado

Ordenado por consecuencia si se rompe.

### 1. Nunca hagas que el descifrado cree una llave

```
final SecretKey? key = await _keyProvider.read(keyId);    // correcto
// await _keyProvider.readOrCreate(...)                 // JAMÁS al descifrar
```

Crearía una llave nueva y dejaría **todos** los registros anteriores ilegibles, en silencio y para siempre.

## 2. Nunca borres el origen antes de verificar el destino

La migración escribe, **comprueba dentro de la misma transacción** y solo después borra. Invertir ese orden es pérdida de datos irreversible.

## 3. Nunca confíes en un campo derivado de un archivo

```
ScanRecord.fromJson(map, trustDerivedAnalysis: false)    // en importación
```

Sin ese `false`, un respaldo manipulado puede declararse `allow` y la interfaz lo mostrará como tal.

## 4. Nunca quites la eliminación de `effectiveUri`

Omitir `rawPayload` no basta: `effectiveUri` lleva la misma consulta y el mismo token. Es lo que separa un paquete redactado de una filtración.

## 5. Nunca elimines la frase de resultado normal

«No se observaron señales locales. Esto no demuestra que el destino sea seguro.»

`verify_rootcause_contract.py` falla si desaparece de la interfaz.

## 6. Cuidado con el ciclo de vida de la cámara

`MobileScanner` solo lo gestiona si él crea el controlador. Aquí lo crean las pantallas, así que **ellas** deben observar `didChangeAppLifecycleState`. Al reiniciar: parar, desechar y **solo entonces** crear el nuevo, porque ambos comparten una única sesión de cámara.

## 7. Cuidado con los `switch` sobre enumeraciones

Dart exige exhaustividad. Añadir un valor a `ScanPhase` o a `ContentKind` obliga a revisar cada `switch`, y son varios repartidos por las pantallas.

## 8. La versión vive en dos sitios que deben coincidir

`pubspec.yaml` y `lib/core/app_info.dart`. El validador falla si divergen.

## Convenciones del código

| Convención                         | Detalle                                                                         |
|------------------------------------|---------------------------------------------------------------------------------|
| Tipos explícitos                   | <code>final String x = ..., &lt;Widget&gt;[...]</code> . Es el estilo dominante |
| <code>final</code> en locales      | Lint <code>prefer_final_locals</code>                                           |
| Sin <code>print</code>             | Lint <code>avoid_print</code> ; además, podría filtrar una carga                |
| <code>unawaited()</code> explícito | Cuando un futuro no se espera a propósito                                       |

|                                 |                                                           |
|---------------------------------|-----------------------------------------------------------|
| on Object catch                 | Con un comentario que explique por qué se ignora          |
| Comentarios de <b>por qué</b>   | No describas la línea: explica la decisión                |
| Documentación en la declaración | /// sobre clases y métodos públicos                       |
| Textos visibles en español      | La interfaz es monolingüe hoy                             |
| Idioma de los comentarios       | Conviven español e inglés; sigue el del archivo que tocas |

## Convenciones de Git

- **una rama por cambio**, con un solo tema;
- mensajes con tipo(ámbito): resumen y un cuerpo que explique la **causa**;
- **nunca git add -A**: rutas explícitas, para no colar artefactos ni datos;
- CHANGELOG.md si el cambio es visible;
- describe qué comandos ejecutaste, y no afirmes que algo pasa sin haberlo visto pasar.

## Antes de abrir un pull request

```
python tool/validate_structure.py --require-lock
python tool/verify_rootcause_contract.py
flutter analyze --fatal-infos
flutter test
```

Si tocaste una regla, además:

```
flutter test test/core/qr_investigation_engine_test.dart
flutter test test/core/qr_evidence_exporter_test.dart
```

## Itinerario sugerido

### Día 1 — leer

Los siete documentos de la lista, y recorre los ocho archivos del flujo principal sin cambiar nada.

### Día 2 — ejecutar

Levanta la aplicación, escanea un QR de `https://example.com` desde el generador, mira el resultado, exporta la evidencia y **léela**. Rompe algo a propósito —cambia el peso de una regla— y observa cuál de los gates te lo dice.

### Día 3 — cambiar algo pequeño

Elige una tarea de la lista de abajo. Hazla completa: código, prueba, documentación.

### Semana 1 — una regla nueva

Propón una regla, discútela y complétala con sus siete pasos. Al terminar, entiendes el producto.

### Semana 2 — un módulo entero

Toma un área con poca cobertura —repositorios, recuperación— y añade pruebas. Aprenderás más leyendo para probar que leyendo para leer.

## Tareas apropiadas para empezar

Ordenadas de menor a mayor dificultad. Las tres primeras cierran hallazgos reales del informe de riesgos.

| # | Tarea                                                                                                                               | Dificultad | Cierra                                   |
|---|-------------------------------------------------------------------------------------------------------------------------------------|------------|------------------------------------------|
| 1 | Prueba que ejecute los 12 fixtures contra el motor y compare severidad, acción, <code>mustInclude</code> y <code>mustExclude</code> | Baja       | <b>R-01</b> , el hallazgo de mayor valor |
| 2 | Pruebas de <code>SettingsRepository</code> : valores por defecto y <code>resetNonSensitive</code>                                   | Baja       | Cobertura                                |
| 3 | Aplicar la ocultación de sensibles también en la lista del historial                                                                | Baja       | <b>R-10</b>                              |
| 4 | Pruebas de <code>ContentInterpreter</code> para AAMVA, Swiss QR y EPC                                                               | Media      | Cobertura de <code>sensitive</code>      |
| 5 | Pruebas de la migración del historial heredado                                                                                      | Media      | <b>R-02</b>                              |
| 6 | Migrar el inventario a <code>MobileScannerEngine</code>                                                                             | Media      | <b>R-08</b>                              |
| 7 | Extraer el filtro de repetición y la decisión de persistencia a funciones puras, y probarlas                                        | Media      | <b>R-06</b>                              |
| 8 | Migrar las cadenas de una pantalla a <code>AppLocalizations</code>                                                                  | Media      | <b>R-13</b>                              |

## A quién preguntar

Un solo mantenedor: [Vladimir Acuña](#). Para vulnerabilidades, el canal privado de `../../SECURITY.md`, sin adjuntar datos reales.

## Errores que ya cometió alguien

Están documentados para que no se repitan:

| Error                                                              | Dónde se cuenta                          |
|--------------------------------------------------------------------|------------------------------------------|
| Fijar la ventana de lectura con un umbral que impedía recalcularla | <code>../quality/SCANNER_UX.md</code> §1 |



|                                                                   |                                       |
|-------------------------------------------------------------------|---------------------------------------|
| Dejar el ciclo de vida de la cámara sin atender                   | Íd.                                   |
| Usar el sonido del sistema, que el usuario suele tener apagado    | Íd. §3                                |
| Anunciar una lectura conseguida con el texto de «en pausa»        | Íd. §6                                |
| Usar el marco como filtro y descartar códigos en silencio         | Íd. §6                                |
| Dejar la resolución de cámara sin declarar: Android cae a 640×480 | Íd. §6                                |
| Mostrar en la interfaz una versión distinta de la construida      | <code>check_interface_version</code>  |
| Sustituir un import por otro que no reexporta el mismo tipo       | <a href="#">14-troubleshooting.md</a> |

Todos comparten un patrón: **un fallo silencioso es peor que uno ruidoso**. Si tu cambio puede hacer que algo no ocurra, haz que la aplicación lo diga.